

Getting started with PyLucene - Indexing

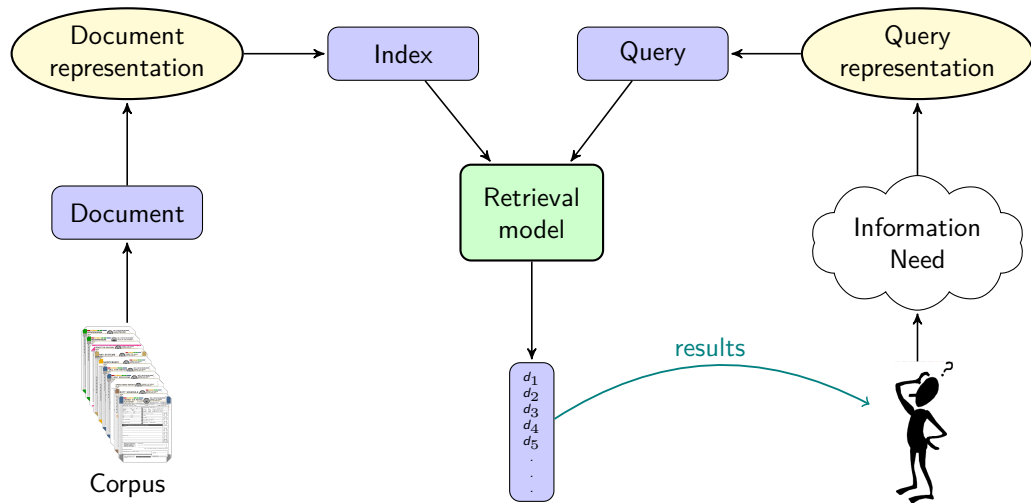
Introduction to Information Retrieval

Dwaipayan Roy

Department of Computational and Data Sciences



Information Retrieval - A Graphical Representation



Popular Retrieval Systems for IR Research

System	Developed By	Initial Release	Language
SMART	Cornell University	1960s1970s	C
Indri	UMass Amherst & CMU (Lemur Project)	2002	C++
Galago	UMass Amherst (Lemur Project)	2011	Java
Terrier	University of Glasgow	2004	Java
Lucene	Doug Cutting (Apache Software Foundation)	19992000	Java

- **Lucene** (written for non-academic purposes)

Popular Retrieval Systems for IR Research

System	Developed By	Initial Release	Language
SMART	Cornell University	1960s1970s	C
Indri	UMass Amherst & CMU (Lemur Project)	2002	C++
Galago	UMass Amherst (Lemur Project)	2011	Java
Terrier	University of Glasgow	2004	Java
Lucene	Doug Cutting (Apache Software Foundation)	19992000	Java

- **Lucene** (written for non-academic purposes)

- ▶ Robustness
- ▶ Flexibility
- ▶ Bigdata handling efficiency

Apache Lucene



- Java-based information retrieval engine.
- Apache Open Source Project.
- Widespread library for full text search.
- Related projects: ElasticSearch, Solr, Tika, Nutch, ...

Apache Lucene



- Java-based information retrieval engine.
- Apache Open Source Project.
- Widespread library for full text search.
- Related projects: Elasticsearch, Solr, Tika, Nutch, ...



Apache Lucene



- Java-based information retrieval engine.
- Apache Open Source Project.
- Widespread library for full text search.
- Related projects: ElasticSearch, Solr, Tika, Nutch, ...



Full list: <https://cwiki.apache.org/confluence/display/lucene/poweredby>

Features of Lucene

- No built-in support for parsing regular file (such as .doc, .pdf etc.)

Features of Lucene

- No built-in support for parsing regular file (such as .doc, .pdf etc.)

Can't be used out of the box!

Features of Lucene

- No built-in support for parsing regular file (such as .doc, .pdf etc.)

Can't be used out of the box!

- Simple API.
- Fast.
- Concurrent indexing and searching.
- A series of implemented retrieval models.
- Free and open-source.

PyLucene - a Python wrapper for Lucene

- Python extension for accessing Java Lucene.
- Goal → allow us to use Lucene's *text indexing* and *searching* capabilities from Python.

PyLucene - a Python wrapper for Lucene

- Python extension for accessing Java Lucene.
- Goal → allow us to use Lucene's *text indexing* and *searching* capabilities from Python.
- Embeds a Java VM with Lucene into a Python process.

PyLucene - a Python wrapper for Lucene

- Python extension for accessing Java Lucene.
- Goal → allow us to use Lucene's *text indexing* and *searching* capabilities from Python.
- Embeds a Java VM with Lucene into a Python process.
- Built with JCC, a C++ code generator that makes it possible to call into Java classes from Python via Java's Native Invocation Interface (JNI).

PyLucene - a Python wrapper for Lucene

- Python extension for accessing Java Lucene.
- Goal → allow us to use Lucene's *text indexing* and *searching* capabilities from Python.
- Embeds a Java VM with Lucene into a Python process.
- Built with JCC, a C++ code generator that makes it possible to call into Java classes from Python via Java's Native Invocation Interface (JNI).
- Sources for JCC are included with the PyLucene sources.

PyLucene - a Python wrapper for Lucene

- Python extension for accessing Java Lucene.
- Goal → allow us to use Lucene's *text indexing* and *searching* capabilities from Python.
- Embeds a Java VM with Lucene into a Python process.
- Built with JCC, a C++ code generator that makes it possible to call into Java classes from Python via Java's Native Invocation Interface (JNI).
- Sources for JCC are included with the PyLucene sources.

We will be using PyLucene version 10.

Installing PyLucene

Prerequisites

- 1 Python version 3.x.
- 2 Java version ≥ 21 .
- 3 Ant and Ivy.
- 4 Download PyLucene from: <https://lucene.apache.org/pylucene/>
- 5 Install JCC.

Installing PyLucene

Prerequisites

- 1 Python version 3.x.
- 2 Java version ≥ 21 .
- 3 Ant and Ivy.
- 4 Download PyLucene from: <https://lucene.apache.org/pylucene/>
- 5 Install JCC.

• Install PyLucene

- ▶ edit MakeFile based on your system's Java installation paths.
- ▶ `make #` this may need sudo permission.
- ▶ `make install #` this could take more than half an hour.
- ▶ `make test #` testing whether the installation is successful or not.

Installing PyLucene

Prerequisites

- 1 Python version 3.x.
- 2 Java version ≥ 21 .
- 3 Ant and Ivy.
- 4 Download PyLucene from: <https://github.com/PyLucene/pylucene/>
- 5 Install JCC.



- Install PyLucene
 - ▶ edit MakeFile based on your system's Java installation paths.
 - ▶ make # this may need sudo permission.
 - ▶ make install # this could take more than half an hour.
 - ▶ make test # testing whether the installation is successful or not.

Installing PyLucene: The Easy Way

- **The Problem:** Installing PyLucene directly can be complex due to Java dependencies.

Installing PyLucene: The Easy Way

- **The Problem:** Installing PyLucene directly can be complex due to Java dependencies.
- **The Solution:** Use *Docker* to automate the setup in an isolated, pre-configured environment.

Installing PyLucene: The Easy Way

- **The Problem:** Installing PyLucene directly can be complex due to Java dependencies.
- **The Solution:** Use *Docker* to automate the setup in an isolated, pre-configured environment.
- **What is Docker?** A platform for developing, shipping, and running applications in lightweight *containers*.
- **Benefit:** A container packages an application with all its dependencies, guaranteeing consistent behavior across any machine.



Installing PyLucene: The Easy Way

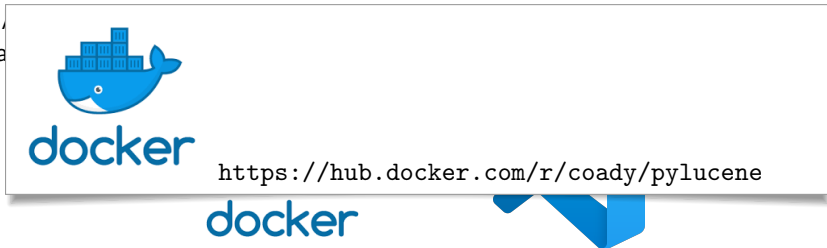
- **The Problem:** Installing PyLucene directly can be complex due to Java dependencies.
- **The Solution:** Use *Docker* to automate the setup in an isolated, pre-configured environment.
- **What is Docker?** A platform for developing, shipping, and running applications in lightweight *containers*.
- **Benefit:** A container packages an application with all its dependencies, guaranteeing consistent behavior across any machine.



- **Our Workflow:** Using a Docker container with PyLucene pre-installed and connect to it using VS Code for a seamless development experience.

Installing PyLucene: The Easy Way

- **The Problem:** Installing PyLucene directly can be complex due to Java dependencies.
- **The Solution:** Use *Docker* to automate the setup in an isolated, pre-configured environment.
- **What is Docker?** A platform for developing, shipping, and running applications in lightweight *containers*.
- **Benefit:** Ensuring consistent behavior across different environments.



- **Our Workflow:** Using a Docker container with PyLucene pre-installed and connect to it using VS Code for a seamless development experience.

Lucene: Documents and Fields

- **Document = atomic unit of indexing and searching.**
 - ▶ Every web page, PDF, email, or database row is represented as a Document.

Lucene: Documents and Fields

- **Document = atomic unit of indexing and searching.**
 - ▶ Every web page, PDF, email, or database row is represented as a Document.
- A Document contains **Fields**.
 - ▶ Each Field has a **name** and a **value**.
 - ▶ Examples of fields:
 - ★ `title, director, plot, ...`
 - ★ `title, author, date, abstract`
 - ★ `body, URL, keywords, ...`
- Different documents can have different sets of Fields.
 - ▶ e.g., a web page may have a URL field but a PDF may not.
- Raw content must be **translated into Fields**.
 - ▶ Example: parse HTML → extract title, body, links.

Lucene: Documents and Fields

- **Document = atomic unit of indexing and searching.**
 - ▶ Every web page, PDF, email, or database row is represented as a Document.
- A Document contains **Fields**.
 - ▶ Each Field has a **name** and a **value**.
 - ▶ Examples of fields:
 - ★ `title`, `director`, `plot`, ...
 - ★ `title`, `author`, `date`, `abstract`
 - ★ `body`, `URL`, `keywords`, ...
- Different documents can have different sets of Fields.
 - ▶ e.g., a web page may have a URL field but a PDF may not.
- Raw content must be **translated into Fields**.
 - ▶ Example: parse HTML → extract title, body, links.
- Querying is field-specific:
 - ▶ Syntax: `field-name:query-term`
 - ▶ Example: `director:nolan`



Lucene: Field Options

- Fields may:
 - ▶ **Be indexed or not**
 - ★ Indexed fields contribute to the inverted index and are searchable.
 - ★ Non-indexed fields are ignored in searches.

Lucene: Field Options

- Fields may:
 - ▶ **Be indexed or not**
 - ★ Indexed fields contribute to the inverted index and are searchable.
 - ★ Non-indexed fields are ignored in searches.
 - ▶ **Be analyzed (tokenized) or not**
 - ★ Analyzed fields are processed by an **Analyzer** (tokenization, stemming, stopword removal).
 - ★ Non-analyzed fields are treated as a single token.
 - ★ Useful for IDs, URLs, dates, paths, SSNs, etc.

Lucene: Field Options

- Fields may:
 - ▶ **Be indexed or not**
 - ★ Indexed fields contribute to the inverted index and are searchable.
 - ★ Non-indexed fields are ignored in searches.
 - ▶ **Be analyzed (tokenized) or not**
 - ★ Analyzed fields are processed by an **Analyzer** (tokenization, stemming, stopword removal).
 - ★ Non-analyzed fields are treated as a single token.
 - ★ Useful for IDs, URLs, dates, paths, SSNs, etc.
 - ▶ **Be stored or not**
 - ★ Stored fields can be retrieved and displayed to users.
 - ★ Not needed for search, but useful for display (e.g., title, snippet).

Lucene: Field Options

- Fields may:

- ▶ **Be indexed or not**

- ★ Indexed fields contribute to the inverted index and are searchable.
 - ★ Non-indexed fields are ignored in searches.

- ▶ **Be analyzed (tokenized) or not**

- ★ Analyzed fields are processed by an **Analyzer** (tokenization, stemming, stopword removal).
 - ★ Non-analyzed fields are treated as a single token.
 - ★ Useful for IDs, URLs, dates, paths, SSNs, etc.

- ▶ **Be stored or not**

- ★ Stored fields can be retrieved and displayed to users.
 - ★ Not needed for search, but useful for display (e.g., title, snippet).

- ▶ **Optionally store term vectors**

- ★ Similar to a positional index per field.
 - ★ Enable advanced features:
 - Highlighting query terms in documents.
 - Finding similar documents.
 - Query expansion.

Why Field Options Matter

- Fields control how data is *indexed*, *stored*, and *queried*.
- Choices affect:
 - ▶ Searchability.
 - ▶ Performance.
 - ▶ Supported features (sorting, faceting, retrieval)
- Matching field type to use-case is essential.

Basic Field Properties (FieldType)

Property	Controls
stored	Value kept and returned in hits
indexed	Content searchable via inverted index
tokenized	Text broken into tokens or kept as single term

Common Field Subclasses in Lucene 10

Field Type	Description / Use Case
<code>TextField</code>	Analyzed, tokenized text. Used for full-text fields such as titles, abstracts, or document body.
<code>StringField</code>	Not tokenized (stored as a single term). Useful for identifiers, tags, categories, or exact matches.
Point Fields (<code>IntField</code> , <code>LongField</code> , <code>FloatField</code> , <code>DoubleField</code>)	Numeric values, support range queries and numeric filtering.
<code>StoredField</code>	Not indexed, only stored. Useful for metadata or original values you want to display back to users (URLs, links).

Lucene Field Options Comparison

Field Type	Indexed	Tokenized	Stored	Term Vectors
<code>TextField</code>	Yes	Yes	Optional	No
<code>StringField</code>	Yes	No	Optional	No
Point Fields	Yes (numeric index)	n/a	Optional	No
<code>StoredField</code>	No	No	Yes	No
<code>Field</code>	Tunable	Tunable	Tunable	Tunable

Lucene `TextField` Syntax

```
TextField(String name, String value, Field.Store store)
```

`TextField` - Creates a new `TextField` with `String` value.

```
public TextField(String name,  
String value,  
Field.Store store)
```

- name - field name
- value - string value
- store - `Store.YES` if the content should also be stored

- **Throws:** `IllegalArgumentException` - if the field name or value is null.

```
TextField("title", "The Prestige", Field.Store.YES)
```

★ Useful for searchable, full-text content.

Lucene `StringField` Syntax

```
StringField(String name, String value, IndexableFieldType type)
```

`StringField` - Creates a new `StringField`, indexing value as a single token.

```
public StringField(String name,  
String value,  
Field.Store stored)
```

- name - field name
- value - string value
- store - `Store.YES` if the content should also be stored

- **Throws:** `IllegalArgumentException` - if the field name or value is null.

```
StringField("origin", "Indian", Field.Store.YES)
```

★ Useful for exact match.

Lucene `IntField` (or `LongField`, `FloatField`, `DoubleField`) Syntax

```
IntField(String name, String value, Field.Store stored)
```

`IntField` - index integers for exact/range queries.

```
public IntField(String name,  
    int value,  
    Field.Store stored)
```

- name - field name
- value - int value
- stored - whether to store the field

- **Throws:** `IllegalArgumentException` - if the field name or value is null.

```
IntField("year", "2008", Field.Store.YES)
```

- ★ stores a per-document int value for scoring, sorting or value retrieval and index the field for fast range filters.

Lucene `StoredField` Syntax

```
StoredField(String name, String value, IndexableFieldType type)
```

`StoredField` - Stored-only value for retrieving in summary results .

```
public StoredField(String name,  
                    String value)
```

- name - field name
- value - string value

- **Throws:** `IllegalArgumentException` - if the field name or value is null.

```
StoredField("url", "https://dwaipayanroy.github.io/", Field.Store.YES)
```

★ Only to store meta-data; **not searchable**.

Common Field Types in Lucene 10

- `TextField` – analyzed, tokenized (full-text content).
- `StringField` – exact matches (IDs, tags).
- **Point Fields** (`IntField`, `LongField`, `FloatField`, `DoubleField`) – numeric data, range queries.
- `StoredField` – retrievable metadata, not searchable.

Lucene 10 Field Types Quick Reference

Field Type	Best For	Storage
<code>TextField</code>	Full-text search, content analysis, tokenized text	Optional
<code>StringField</code>	IDs, categories, exact matching, faceting	Optional
Point Fields	Numeric ranges, spatial data, performance queries	Optional
<code>StoredField</code>	Display values, retrieval-only data	Always stored

Lucene 10 Field Types with Examples

Field Type	Best For	Example Usage
<code>TextField</code>	Full-text search, content analysis	<pre>TextField("content", "The quick brown fox", Field.Store.YES)</pre>
<code>StringField</code>	IDs, categories, exact matching	<pre>StringField("product_id", "P12345", Field.Store.YES)</pre>
Point Fields	Numeric ranges, spatial data	<pre>IntField("price", 2999, Field.Store.YES)</pre>
<code>StoredField</code>	Display values, retrieval-only	<pre>StoredField("image_url", "https://example.com/img.jpg")</pre>

Lucene 10 Field Types with Examples

Field Type	Best For	Example Usage
<code>TextField</code>	Full-text search, content analysis	<pre>TextField("content", "The quick brown fox", Field.Store.YES)</pre>
<code>StringField</code>	IDs, categories, exact matching	<pre>StringField("id", "12345", Field.Store.YES)</pre>
Point Fields	Numeric ranges, spatial data	<pre>IntField("price", 2999, Field.Store.YES)</pre>
<code>StoredField</code>	Display values, retrieval-only	<pre>StoredField("image_url", "https://example.com/img.jpg")</pre>

None of these fields store the accessible vectors!

Lucene Field Syntax

```
Field(String name, CharSequence value, IndexableFieldType type)
```

Field - Create a field with String value.

```
public Field(String name,  
             CharSequence value,  
             IndexableFieldType type)
```

- name - field name
- value - string value
- type - field type

- **Throws:** `IllegalArgumentException` - if either the name, value or type is null, or if the field's type is neither `indexed()` nor `stored()`, or if `indexed()` is false but `storeTermVectors()` is true.

```
Field("title", "The Dark Knight", indexType)
```

Defining indexType

```
indexType = FieldType()
indexType.setStored(True)
indexType.setTokenized(True)
indexType.setStoreTermVectors(True)
indexType.setIndexOptions(IndexOptions.DOCS_AND_FREQS_AND_POSITIONS)
indexType.setStoreTermVectorPositions(True) # Explicitly store positions
indexType.setStoreTermVectorOffsets(False) # Char. offset not
```

DOCS	Only documents are indexed: term frequencies, positions are omitted.
DOCS_AND_FREQS	Only documents and term frequencies are indexed: positions omitted.
DOCS_AND_FREQS_AND_POSITIONS (default)	Indexes documents, frequencies, and positions.
DOCS_AND_FREQS_AND_POSITIONS_AND_OFFSETS	Indexes documents, frequencies, positions, and offsets.
NONE	Not indexed.

Defining indexType

```
indexType = FieldType()  
indexType.setStored(True)  
indexType.setTokenized(True)  
indexType.setStoreTermVectors(True)  
indexType.setIndexOptions(IndexOptions.DOCS_AND_FREQS_AND_POSITIONS)  
indexType.setStoreTermVectorPositions(True) # Explicitly store positions  
indexType.setStoreTermVectorOffsets(False) # Char. offset not
```

DOCS	Only documents are indexed: term frequencies, positions are omitted.
DOCS_AND_FREQS	Only documents and term frequencies are indexed: positions omitted.
DOCS_AND_FREQS_AND_POSITIONS (default)	Indexes documents, frequencies, and positions.
DOCS_AND_FREQS_AND_POSITIONS_AND_OFFSETS	Indexes documents, frequencies, positions, and offsets.
NONE	Not indexed.

- ★ DOCS_AND_FREQS - Adds frequency data (better scoring).
- ★ DOCS_AND_FREQS_AND_POSITIONS - Enables phrase queries (most common).

Possible Confusion

Storing positions!

I thought `indexType.setIndexOptions(IndexOptions.DOCS_AND_FREQS_AND_POSITIONS)` is storing the positions as well. Is this position and `indexType.setStoreTermVectorPositions(True)` different?

Possible Confusion

Storing positions!

I thought `indexType.setIndexOptions(IndexOptions.DOCS_AND_FREQS_AND_POSITIONS)` is storing the positions as well. Is this position and `indexType.setStoreTermVectorPositions(True)` different?

✍ **Yes, they are completely different things!** This is a common point of confusion in Lucene.

- `setIndexOptions(IndexOptions.DOCS_AND_FREQS_AND_POSITIONS)`
 - ▶ Purpose: Stores positions in the main inverted index for searching
 - ▶ Used for: Phrase queries, proximity searches, span queries
 - ▶ Example: Enables queries like `"quick brown fox" 5`
 - ▶ Access: *Used internally* by Lucene's search algorithms
- `setStoreTermVectorPositions(True)`
 - ▶ Purpose: Stores positions in term vectors for retrieval and analysis
 - ▶ Used for: Getting position data back out for analysis, highlighting, etc.
 - ▶ Example: Used by FastVectorHighlighter, custom analysis tools
 - ▶ Access: Available via `IndexReader.getTermVectors()`

PyLucene document building

academic papers

Relevance-Based Language Models

Victor Lavrenko and W. Bruce Croft
Center for Intelligent Information Retrieval
Department of Computer Science
University of Massachusetts, Amherst, MA 01003
{lavrenko@cs.umass.edu, croft@cs.umass.edu}

ABSTRACT

We explore the relation between classical probabilistic models of information retrieval and the emerging language modeling approaches. It has long been recognized that the primary obstacle to effective performance of classical models is the need to estimate a relevance-weighted probability of words in the relevant class. We propose a novel technique for estimating these probabilities using the query alone. We demonstrate that our technique can produce highly accurate relevance models, achieving important advances in synonymy and polysemy. Our experiments show relevance models incorporating flexible language modeling systems on TREAC, TREC and TDT tracking tasks. The main contribution of this work is an effective formal method for estimating a relevance model with no training data.

1. INTRODUCTION

Recent developments in the conceptual view of Information Retrieval method diverges from the traditional models of relevance and the emergence of language modeling frameworks, introduced by Pinski and Croft [20]. A number of following publications [2, 14, 23, 18], adopted a similar framework, introducing refinements in parameter estimation, the use of multi-word features and expansion techniques. A common theme in these models is that they do not exploit models of relevance, instead attempting to model the query generation process. These approaches model query generation as random sampling from one of the document models. During the relevance process, the documents are ranked by the probability that queries would be observed as a random sample from the respective document model.

Earlier work on probabilistic models of information retrieval [19, 16, 17, 22] took a conceptually different approach. Relevance was explicitly represented in model word occurrences in relevant and non-relevant classes of documents, and statistical models to classify the document into the more likely class. For retrieval purposes, the documents were ranked by the probability that they belong to the relevant class.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are made or distributed for profit or commercial advantage and that copies bear the name and address of the author(s) and the publisher. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or fee.
SIGIR '04, September 5-9, 2004, New Orleans, Louisiana, USA.
Copyright 2004 ACM 978-1-58113-324-9/04/0009...\$5.00.

The difference between the two approaches is not superficial, but is in fact quite important. Traditional probabilistic frameworks allow for relatively rich and complex query models (e.g. [22]), whereas the recent language modeling approaches treat the query as a fixed sample of text with little room for modeling. Many popular techniques in Information Retrieval, such as relevance feedback and automatic query expansion have a very intuitive interpretation in the traditional probabilistic models, but are conceptually difficult to integrate into the language modeling framework of [18] because they involve augmenting the sample text (13) rather than adjusting the model. Furthermore, explicit models of relevance are better suited to other information organization tasks, such as collaborative [6], question answering [8], topic detection and tracking (TDT) [17, 15] and text representation [3].

The primary obstacle to constructing effective models of relevance is the lack of training data. In a typical retrieval environment we are given a query, a large collection of documents and no indication of which documents might be relevant. In this paper we introduce a technique for constructing a relevance model from the query alone and compare resulting query-based relevance models to language models constructed with training data. We also evaluate the retrieval effectiveness of query-based relevance models on two-domain tasks: the ad hoc retrieval task of TREAC [2], and the tracking task of TDT [23]. The remainder of the paper is organized as follows. We briefly survey the classical relevance-based probabilistic models as well as other non-language modeling approaches in section 2. Our method for formally constructing and applying relevance models is detailed in section 3. Section 4 provides a thorough evaluation of query-based relevance models in terms of cross-corpus, retrieval effectiveness and tracking accuracy.

2. RELATED WORK

There are two domains of information retrieval research that provide a theoretical foundation for our model: the now classic work on probabilistic models of relevance, and the recent developments in language modeling techniques for IR. In the former we review the concept of a relevance model as a language model representative of a class of relevant documents. To the latter we outline the methods of modeling and specific estimation techniques. In the section we give a brief survey of several developments in both of these directions, highlighting promising connections between the two.

2.1 Classical Probabilistic Models

Underlying most research on probabilistic models of Information Retrieval is the probability matching principle, detailed by Robertson [19], which suggests ranking the documents D by the odds of their being observed in the relevant class: $P(L|R)/P(L|\bar{R})$.

- Collection: set of *documents* - academic papers.
- Document: set of *fields* - containing different sections of the paper.

PyLucene document building

academic papers

Relevance-Based Language Models

Victor Lavrenko and W. Bruce Croft
Center for Intelligent Information Retrieval
Department of Computer Science
University of Massachusetts, Amherst, MA 01003
{lavrenko|croft}@cs.umass.edu

ABSTRACT

We explore the relation between classical probabilistic models of information retrieval and the emerging language modeling approaches. It has long been recognized that the primary obstacle to effective performance of classical models is the need to estimate a relevance model: probabilities of words in the relevant class. We propose a novel technique for estimating these probabilities using the query alone. We demonstrate that this technique can produce highly accurate relevance models, addressing important sources of synonymy and polysemy. Our experiments show relevance models improving existing machine language modeling systems on TREC, national and TDT tracking tasks. The main contribution of this work is an effective formal method for estimating a relevance model with no training data.

1. INTRODUCTION

Recent developments in the conceptual views of Information Retrieval method diverges from the traditional models of relevance and the emergence of language modeling frameworks, introduced by Pinski and Croft [20]. A number of Information Retrieval [1, 14, 15, 18], adopted a similar framework, introducing relevance in probabilistic estimation, the word models used feature and expansion techniques. A common theme in these models is that they describe explicit models of relevance, instead attempting to model the query generation process. These approaches model query generation in random sampling from one of the document models. During the relevance process, the documents are ranked by the probability that a query would be observed in a random sample from the respective document model.

Earlier work on probabilistic models of information retrieval [19, 16, 17, 22] took a conceptually different approach. Relevance is explicitly interpreted as model word occurrences in relevant and non-relevant classes of documents, and statistical models to classify the document into the more likely class. For retrieval purposes, the documents were ranked by the probability that they belong to the relevant class.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are made or distributed for profit or commercial advantage and that copies bear the name and address of the author(s) and the publisher. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or fee.
SIGIR '04, September 5-9 2004, New Orleans, Louisiana, USA.
Copyright 2004 ACM 978-1-58113-326-0/04/0009...\$5.00.

The difference between the two approaches is not superficial, but is in fact quite important. Traditional probabilistic frameworks allow for relatively rich and complex query models (e.g. [22]), whereas the recent language modeling approaches treat the query as a fixed sample of text with little room for modeling. Many popular techniques in Information Retrieval, such as relevance feedback and automatic query expansion have a very intuitive interpretation in the traditional probabilistic models, but are conceptually difficult to integrate into the language modeling framework of [19] because they involve augmenting the sample text [17] rather than adjusting the model. Furthermore, explicit models of relevance are better suited to other information representation tasks, such as compression [16], question answering [14], topic detection and tracking (TDT) [17, 15] and text representation [16].

The primary obstacle to constructing effective models of relevance is the lack of training data. In a typical retrieval environment we are given a query, a large collection of documents and no indication of which documents might be relevant. In this paper we introduce a technique for constructing a relevance model from the query alone and compare resulting query-based relevance models to language models constructed with training data. We also evaluate the retrieval effectiveness of query-based relevance models on two dataset tasks: the ad hoc retrieval task of TREC [5], and the tracking task of TDT [23]. The remainder of the paper is organized as follows. We briefly survey the classical relevance-based probabilistic models as well as the emerging language modeling approaches in section 2. Our method for formally constructing and applying relevance models is detailed in section 3. Section 4 provides a thorough evaluation of query-based relevance models in terms of compression, relevance of the documents and tracking accuracy.

2. RELATED WORK

There are two categories of information retrieval models that provide a theoretical foundation for our model: the more classic word probabilistic models of relevance, and the more recent developments in language modeling techniques for IR. In the former we view the concept of a relevance model as a large model approximation of a class of relevant documents. To the latter we add the methods of modeling and specific estimation techniques. In the next two sections we give a brief survey of several developments in both of these directions, highlighting promising research directions for the future.

2.1 Classical Probabilistic Models

Underlying most research on probabilistic models of Information Retrieval is the probability matching principle, detailed by Robertson [19], which suggests ranking the documents D by the odds of their being observed in the relevant class, $P(LA|D)/P(ND|D)$.

- Collection: set of *documents* - academic papers.
- Document: set of *fields* - containing different sections of the paper.
 - ▶ title: Relevance based Language Model
 - ▶ authors: Victor Lavrenko, Bruce Croft
 - ▶ publication-venue: SIGIR
 - ▶ year: 2001
 - ▶ pages: 120 - 127
 - ▶ keywords: information system, information retrieval, evaluation
 - ▶ abstract: We explore the relation between probabilistic models of information retrieval ...

PyLucene document building

movie information



Theatrical release poster

- Collection: set of *documents* - movie information.
- Document: set of *fields* - containing different information on the movie.

Directed by [Christopher Nolan](#)

Story by Christopher Nolan
[David S. Goyer](#)

Produced by [Emma Thomas](#)
[Charles Roven](#)
Christopher Nolan

Starring [Christian Bale](#)
[Michael Caine](#)
[Heath Ledger](#)
[Gary Oldman](#)
[Aaron Eckhart](#)
[Maggie Gyllenhaal](#)
[Morgan Freeman](#)

PyLucene document building

movie information



Theatrical release poster

Directed by [Christopher Nolan](#)

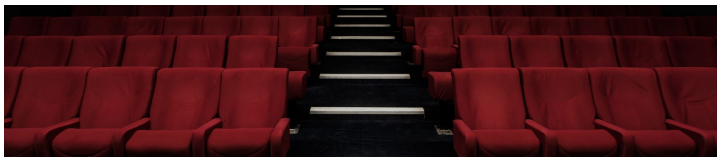
Story by [Christopher Nolan](#)
[David S. Goyer](#)

Produced by [Emma Thomas](#)
[Charles Roven](#)
[Christopher Nolan](#)

Starring [Christian Bale](#)
[Michael Caine](#)
[Heath Ledger](#)
[Gary Oldman](#)
[Aaron Eckhart](#)
[Maggie Gyllenhaal](#)
[Morgan Freeman](#)

- Collection: set of *documents* - movie information.
- Document: set of *fields* - containing different information on the movie.
 - ▶ title: The Dark Knight
 - ▶ release year: 2008
 - ▶ origin: American
 - ▶ director: Christopher Nolan
 - ▶ cast: Christian Bale, Heath Ledger, Michael Caine, Gary Oldman, Morgan Freeman...
 - ▶ genre: superhero
 - ▶ plot: A gang of criminals rob a Gotham City mob bank, murdering each other until only the mastermind remains: the Joker, who escapes with the money...
 - ▶ wikipedia: [https://en.wikipedia.org/wiki/The_Dark_Knight_\(film\)](https://en.wikipedia.org/wiki/The_Dark_Knight_(film))

Indexing the **Wikipedia Movie Plot** Dataset



['Release Year', 'Title', 'Origin/Ethnicity', 'Director', 'Cast', 'Genre', 'Wiki Page', 'Plot']

Dataset source: <https://www.kaggle.com/datasets/jrobischon/wikipedia-movie-plots>

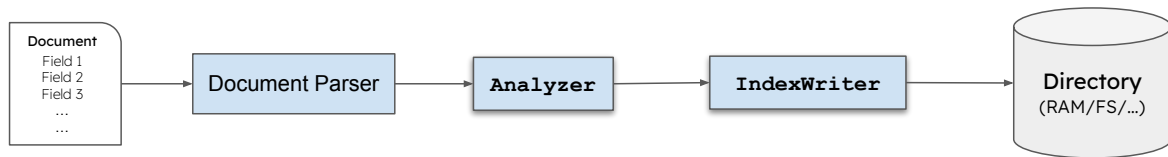
Information Retrieval using PyLucene

- ① Indexing a collection.
- ② Searching in a collection.

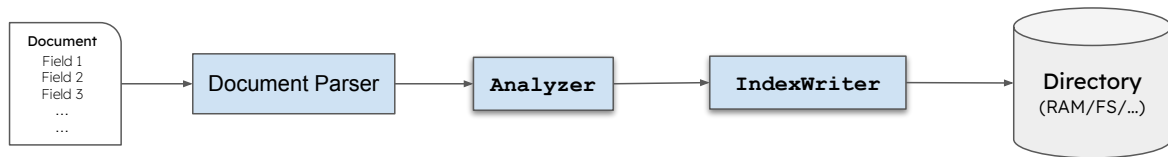
Starting with PyLucene

```
import lucene  
  
lucene.initVM()
```

Indexing using PyLucene



Indexing using PyLucene



- 0 Document parsing.
- 1 Setting and applying analyzer.
- 2 Using `IndexWriter` to write the document in the index.

Indexing in PyLucene:

Document Parser



- HTML
- PDF
- CSV TSV
- TREC
- WARC
- JSON
- ...

- We have to write/use parser based on the data we will be handling.

Indexing in PyLucene

parsing CSV files using pandas



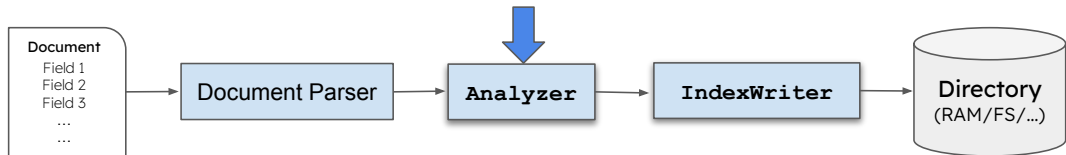
```
import pandas as pd

def read_csv_File(file_path):
    df = pd.read_csv(file_path)
    docid = 0

    for ind in df.index:
        print("{} - {}".format(docid, df['Title'][ind]))
        docid += 1
```

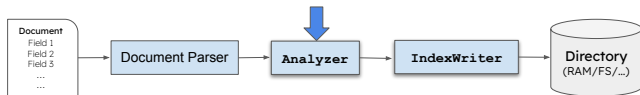
- The docid here is just for printing a structured output.
- PyLucene sets and uses its own docid internally.
- Document can be accessed based on this internal docid.

Indexing in PyLucene



- 0 Document parsing.
- 1 **Setting and applying analyzer.**
- 2 Using IndexWriter to write the document in the index.

Indexing in PyLucene

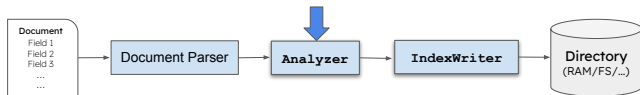


- Input: set of documents (corpus) to index;
- Output: index of documents.

```
import lucene

from org.apache.lucene.analysis.standard import StandardAnalyzer
from org.apache.lucene.document import Document, Field, StringField, FieldType
from org.apache.lucene.index import IndexWriter, IndexWriterConfig
from org.apache.lucene.store import FSDirectory
```

Indexing in PyLucene

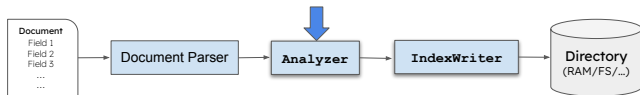


- Input: set of documents (corpus) to index; **language**;
- Output: index of documents.

```
import lucene

from org.apache.lucene.analysis.standard import StandardAnalyzer
from org.apache.lucene.document import Document, Field, StringField, FieldType
from org.apache.lucene.index import IndexWriter, IndexWriterConfig
from org.apache.lucene.store import FSDirectory
```

Indexing in PyLucene



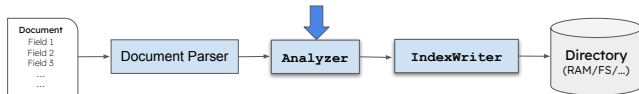
- Input: set of documents (corpus) to index; **language**; **stopword list**;
- Output: index of documents.

```
import lucene

from org.apache.lucene.analysis.standard import StandardAnalyzer
from org.apache.lucene.document import Document, Field, StringField, FieldType
from org.apache.lucene.index import IndexWriter, IndexWriterConfig
from org.apache.lucene.store import FSDirectory
```

Tokenization using PyLucene

standard analyzer example



```
from java.io import StringReader
from org.apache.lucene.analysis.tokenattributes import CharTermAttribute
from org.apache.lucene.analysis.standard import StandardAnalyzer

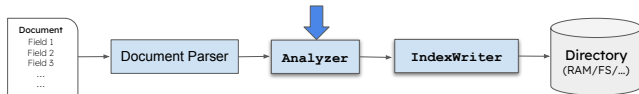
test = "This is how we do it."
analyzer = StandardAnalyzer()

stream = analyzer.tokenStream("", StringReader(test))
stream.reset()
tokens = []

while stream.incrementToken():
    tokens.append(stream.getAttribute(CharTermAttribute.class_).toString())
print(tokens)
```

Analysis using PyLucene

english analyzer example



```
from java.io import StringReader
from org.apache.lucene.analysis.tokenattributes import CharTermAttribute
from org.apache.lucene.analysis.en import EnglishAnalyzer

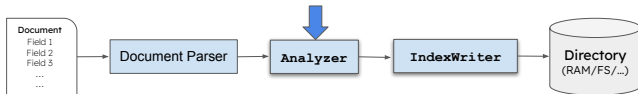
test = "This is how we do it."
analyzer = EnglishAnalyzer()

stream = analyzer.tokenStream("", StringReader(test))
stream.reset()
tokens = []

while stream.incrementToken():
    tokens.append(stream.getAttribute(CharTermAttribute.class_).toString())
print(tokens)
```

Indexing in PyLucene

Analyzer



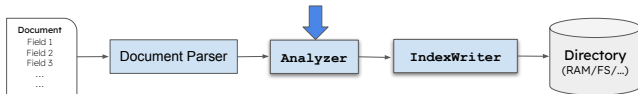
```
# in general:
# import org.apache.lucene.analysis.bn.LangAnalyzer;
# analyzer = new LanguageAnalyzer();

# German analyzer
from org.apache.lucene.analysis.de import GermanAnalyzer
analyzer = new GermanAnalyzer();

# English analyzer
from org.apache.lucene.analysis.en import EnglishAnalyzer
analyzer = new EnglishAnalyzer();
```

Indexing in PyLucene

Analyzer



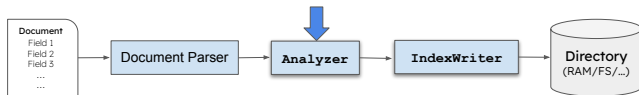
```
# in general:
# import org.apache.lucene.analysis.bn.LangAnalyzer;
# analyzer = new LanguageAnalyzer();

# German analyzer
from org.apache.lucene.analysis.de import GermanAnalyzer
analyzer = new GermanAnalyzer();

# English analyzer
from org.apache.lucene.analysis.en import EnglishAnalyzer
analyzer = new EnglishAnalyzer();
```

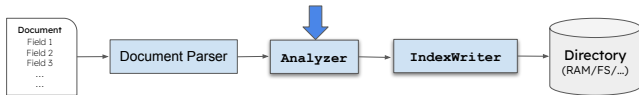
- Set analyzer depending on the **language**.

PyLucene Analyzers



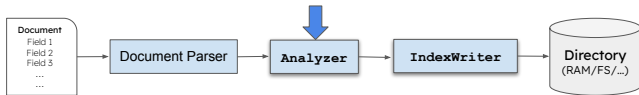
- Process text and prepare it for indexing and searching.
 - ▶ Tokenization.
 - ▶ Lowercasing.
 - ▶ Removing stop words.
 - ▶ Stemming.

PyLucene Analyzers



- Process text and prepare it for indexing and searching.
 - ▶ Tokenization.
 - ▶ Lowercasing.
 - ▶ Removing stop words.
 - ▶ Stemming.
- Common Analyzers that we will be using:
 - ▶ **WhitespaceAnalyzer**: Splits tokens on whitespace.
 - ▶ **StandardAnalyzer**: Does lower-casing and optional stop-word filtering.

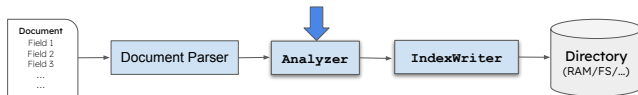
PyLucene Analyzers



- Process text and prepare it for indexing and searching.
 - ▶ Tokenization.
 - ▶ Lowercasing.
 - ▶ Removing stop words.
 - ▶ Stemming.
- Common Analyzers that we will be using:
 - ▶ **WhitespaceAnalyzer**: Splits tokens on whitespace.
 - ▶ **StandardAnalyzer**: Does lower-casing and optional stop-word filtering.
 - ▶ **EnglishAnalyzer**: Filters English stopwords before applying English stemming.

PyLucene Analyzers

example

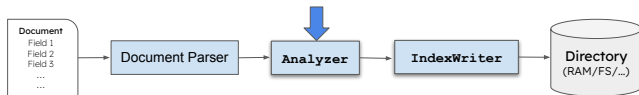


“The quick brown fox jumped over the lazy dog”

- **WhitespaceAnalyzer** :
 - ▶ [The] [quick] [brown] [fox] [jumped] [over] [the] [lazy] [dog]
- **StandardAnalyzer** :
 - ▶ [the] [quick] [brown] [fox] [jumped] [over] [the] [lazy] [dog]

PyLucene Analyzers

example

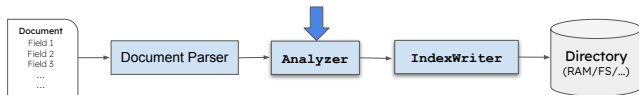


“The quick brown fox jumped over the lazy dog”

- **WhitespaceAnalyzer** :
 - ▶ [The] [quick] [brown] [fox] [jumped] [over] [the] [lazy] [dog]
- **StandardAnalyzer** :
 - ▶ [the] [quick] [brown] [fox] [jumped] [over] [the] [lazy] [dog]
- **EnglishAnalyzer** :
 - ▶ [quick] [brown] [fox] [jump] [over] [lazi] [dog]

PyLucene Analyzers

example



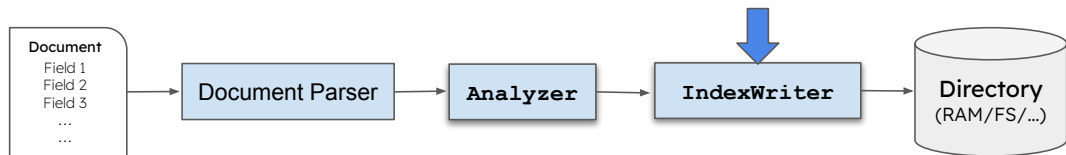
“The quick brown fox jumped over the lazy dog”

- **WhitespaceAnalyzer** :
 - ▶ [The] [quick] [brown] [fox] [jumped] [over] [the] [lazy] [dog]
- **StandardAnalyzer** :
 - ▶ [the] [quick] [brown] [fox] [jumped] [over] [the] [lazy] [dog]
- **EnglishAnalyzer** :
 - ▶ [quick] [brown] [fox] [jump] [over] [lazi] [dog]
- How to remove a set of defined stopwords using either of StandardAnalyzer or EnglishAnalyzer?

Comparison between StandardAnalyzer and EnglishAnalyzer

Feature	StandardAnalyzer	EnglishAnalyzer
Main Purpose	General-purpose analyzer	Optimized for English language
Stopword Removal	Yes (Default: no stopword removal): tunable	Yes (Default: English stopword list): tunable
Stemming	No	Yes (uses <i>Porter Stemmer</i>)
Possessive Filter	No	Yes (e.g., "John's" → "John")
Case Normalization	Yes	Yes
Tokenizer Used	Standard Tokenizer	Standard Tokenizer
Language Specific	No	Yes (English)

Indexing in PyLucene



- 0 Document parsing.
- 1 Setting and applying analyzer.
- 2 Using **IndexWriter** to write the document in the index.

Indexing in PyLucene

IndexWriter



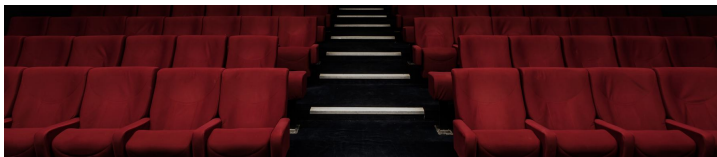
```
from org.apache.lucene.index import IndexWriter, IndexWriterConfig
from org.apache.lucene.store import FSDirectory

indexPath = File("index/").toPath() #from java.io import File
indexDir = FSDirectory.open(indexPath) # RAMDirectory, DbDirectory, FileSwitchDirectory etc.

writerConfig = IndexWriterConfig(StandardAnalyzer())

writer = IndexWriter(indexDir, writerConfig)
```

Indexing the **Wikipedia Movie Plot** Dataset



['Release Year', 'Title', 'Origin/Ethnicity', 'Director', 'Cast', 'Genre', 'Wiki Page', 'Plot']

Dataset source: <https://www.kaggle.com/datasets/jrobischon/wikipedia-movie-plots>

Indexing the Wikipedia movie plot dataset

index the collection

- For each document of the collection:

```
from org.apache.lucene import analysis, document, index, queryparser, search, store, util
import sys, os
from java.nio.file import Paths

def indexDocument(movie):
    fields[] = parseTSVformat(movie)
    doc = document.Document()
    #
    # Index and Store the desired fields with desired options.
    #
    writer.addDocument(doc)
```

Indexing the Wikipedia movie plot dataset

index the collection

- For each document of the collection:

```
from org.apache.lucene import analysis, document, index, queryparser, search, store, util
import sys, os
from java.nio.file import Paths

def indexDocument(movie):
    fields[] = parseTSVformat(movie)
    doc = document.Document()
    #
    # Index and Store the desired fields with desired options.
    #
    writer.addDocument(doc)
```

```
def closeIndexWriter():
    writer.close()
```